NumPy

```
x = [1,2,3,4]
y = [10, 11, 12, 13]
```

```
x
```

```
[1, 2, 3, 4]
```

```
x + y
```

```
[1, 2, 3, 4, 10, 11, 12, 13]
```

```
x * y
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-118-e32109319f52> in <cell line: 1>()
----> 1 x * y

TypeError: can't multiply sequence by non-int of type 'list'
```

```
import numpy as np
```

```
x1 = np.array(x)
y1 = np.array(y)
```

```
x1
```

```
array([1, 2, 3, 4])
```

```
y1
```

```
array([10, 11, 12, 13])
```

```
type(x1)
```

```
numpy.ndarray
```

```
x1 + y1
```

```
array([11, 13, 15, 17])
```

```
x1 * y1
```

```
array([10, 22, 36, 52])
```

```
y1 ** x1
```

```
array([   10,   121,  1728, 28561])
```

```
print(x)
print(x1)
```

```
[1, 2, 3, 4]
[1 2 3 4]
```

```
a1 = np.array([[1, 5, 7, 2], [2, 5, 6, 7]])
```

```
a1
```

```
array([[1, 5, 7, 2],
       [2, 5, 6, 7]])
```

```
print(a1)
```

```
[[1 5 7 2]
 [2 5 6 7]]
```

```
np.zeros((10))
```

```
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
np.zeros((2,3))
```

```
array([[0., 0., 0.],
       [0., 0., 0.]])
```

```
np.zeros((2,3, 2))
```

```
array([[[0., 0.],
        [0., 0.],
        [0., 0.]],

       [[0., 0.],
```

```
        [0., 0.],
        [0., 0.]]])
```

```
np.ones((2,2))
```

```
array([[1., 1.],
       [1., 1.]])
```

```
np.empty((2,2))
```

```
array([[1., 1.],
       [1., 1.]])
```

```
np.empty((2,2), dtype = int)
```

```
array([[4607182418800017408, 4607182418800017408],
       [4607182418800017408, 4607182418800017408]])
```

```
np.arange(5)
```

```
array([0, 1, 2, 3, 4])
```

```
list(range(5))
```

```
[0, 1, 2, 3, 4]
```

```
np.arange(1,5)
```

```
array([1, 2, 3, 4])
```

```
np.arange(2,10)
```

```
array([2, 3, 4, 5, 6, 7, 8, 9])
```

```
a2 = np.arange(12)
```

```
a2
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

```
a2.reshape(4,3)
```

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

```
a2.reshape(2, 2, 3)
```

```
array([[[ 0,  1,  2],
        [ 3,  4,  5]],

       [[ 6,  7,  8],
        [ 9, 10, 11]]])
```

```
a2 = a2.reshape(4,3)
```

```
a2
```

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

```
np.linspace(10, 0.2, 11)
```

```
array([10.  ,  9.02,  8.04,  7.06,  6.08,  5.1 ,  4.12,  3.14,  2.16,
        1.18,  0.2 ])
```

```
np.linspace(start = 10, stop = 11, num = 50)
```

```
array([10.        , 10.02040816, 10.04081633, 10.06122449, 10.08163265,
       10.10204082, 10.12244898, 10.14285714, 10.16326531, 10.18367347,
       10.20408163, 10.2244898 , 10.24489796, 10.26530612, 10.28571429,
       10.30612245, 10.32653061, 10.34693878, 10.36734694, 10.3877551 ,
       10.40816327, 10.42857143, 10.44897959, 10.46938776, 10.48979592,
       10.51020408, 10.53061224, 10.55102041, 10.57142857, 10.59183673,
       10.6122449 , 10.63265306, 10.65306122, 10.67346939, 10.69387755,
       10.71428571, 10.73469388, 10.75510204, 10.7755102 , 10.79591837,
       10.81632653, 10.83673469, 10.85714286, 10.87755102, 10.89795918,
       10.91836735, 10.93877551, 10.95918367, 10.97959184, 11.        ])
```

```
np.linspace(10, 11)
```

```
array([10.        , 10.02040816, 10.04081633, 10.06122449, 10.08163265,
       10.10204082, 10.12244898, 10.14285714, 10.16326531, 10.18367347,
       10.20408163, 10.2244898 , 10.24489796, 10.26530612, 10.28571429,
       10.30612245, 10.32653061, 10.34693878, 10.36734694, 10.3877551 ,
       10.40816327, 10.42857143, 10.44897959, 10.46938776, 10.48979592,
       10.51020408, 10.53061224, 10.55102041, 10.57142857, 10.59183673,
       10.6122449 , 10.63265306, 10.65306122, 10.67346939, 10.69387755,
       10.71428571, 10.73469388, 10.75510204, 10.7755102 , 10.79591837,
       10.81632653, 10.83673469, 10.85714286, 10.87755102, 10.89795918,
       10.91836735, 10.93877551, 10.95918367, 10.97959184, 11.        ])
```

```
np.linspace(10, 11, 5)
```

```
array([10.  , 10.25, 10.5 , 10.75, 11.  ])
```

```
a2
```

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

```
a2.ndim
```

```
2
```

```
a2.reshape(2,2,3).ndim
```

```
3
```

```
a2.shape
```

```
(4, 3)
```

```
a2.reshape(2,2,3).shape
```

```
(2, 2, 3)
```

```
a2.size
```

```
12
```

```
a2.dtype
```

```
dtype('int64')
```

```
np.linspace(10, 11).dtype
```

```
dtype('float64')
```

```
a2.reshape(2,2,3)
```

```
array([[[ 0,  1,  2],
        [ 3,  4,  5]],

       [[ 6,  7,  8],
        [ 9, 10, 11]]])
```

```
a2
```

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

```
True & False
```

```
False
```

```
True & True
```

```
True
```

```
False & False
```

```
False
```

```
True | True
```

```
True
```

```
False | False
```

```
False
```

```
a = np.arange(10)
```

```
a
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
2 * a
```

```
array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])
```

```
a = a + 1
```

```
a
```

```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

```
np.add(a, 1)
```

```
array([ 2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

```
a**0
```

```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

```
a**2 + a
```

```
array([  2,   6,  12,  20,  30,  42,  56,  72,  90, 110])
```

```
a + a
```

```
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18, 20])
```

```
a
```

```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

```
b = np.arange(12)
```

```
b
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

```
a + b
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-182-bd58363a63fc> in <cell line: 1>()
----> 1 a + b

ValueError: operands could not be broadcast together with shapes (10,) (12,)
```

```
a = a*2
```

```
a
```

```
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18, 20])
```

```
b = 2*a
```

```
b
```

```
array([ 4,  8, 12, 16, 20, 24, 28, 32, 36, 40])
```

```
a <= b
```

```
array([ True,  True,  True,  True,  True,  True,  True,  True,  True,
        True])
```

```
a == b
```

```
array([False, False, False, False, False, False, False, False, False,
       False])
```

```
a < 7
```

```
array([ True,  True,  True, False, False, False, False, False, False,
       False])
```

```
a == 10
```

```
array([False, False, False, False,  True, False, False, False, False,
       False])
```

```
a
```

```
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18, 20])
```

```
a[0]
```

```
2
```

```
a2 = a2.reshape(3,4)
```

```
a2
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

```
a2[0, 1]
```

```
1
```

```
a2[:, 0]
```

```
array([0, 4, 8])
```

```
a2[1, :]
```

```
array([4, 5, 6, 7])
```

```
a2[:2, 1]
```

```
array([1, 5])
```

```
a2[1, :2]
```

```
array([4, 5])
```

```
a2[1, 0:2]
```

```
array([4, 5])
```

```
a2[1, 1:3]
```

```
array([5, 6])
```

```
a2[2, 1:]
```

```
array([ 9, 10, 11])
```

```
a2[3,:]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-205-ee2445f35e67> in <cell line: 1>()
----> 1 a2[3,:]

IndexError: index 3 is out of bounds for axis 0 with size 3
```

```
a2[1,:6]
```

```
array([4, 5, 6, 7])
```

```
a2[1,:]
```

```
array([4, 5, 6, 7])
```

```
a2[1,6]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-208-dbc746262d23> in <cell line: 1>()
----> 1 a2[1,6]

IndexError: index 6 is out of bounds for axis 1 with size 4
```

```
a < 5
```

```
array([ True,  True, False, False, False, False, False, False, False,
       False])
```

```
a[a<5]
```

```
a[a==10]
```

```
array([10])
```

```
a
```

```
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18, 20])
```

```
b = a.copy()
```

```
b
```

```
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18, 20])
```

```
a.argmax()
```

```
9
```

```
a.argmin()
```

```
0
```

```
a.min()
```

```
2
```

```
a.max()
```

```
20
```

```
h = a[a>10]
```

```
h
```

```
array([12, 14, 16, 18, 20])
```

```
a
```

```
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18, 20])
```

```
a>20
```

```
array([False, False, False, False, False, False, False, False, False,
       False])
```

```
np.log(a)
```

```
array([0.69314718, 1.38629436, 1.79175947, 2.07944154, 2.30258509,
       2.48490665, 2.63905733, 2.77258872, 2.89037176, 2.99573227])
```

```
np.mean(a)
```

```
11.0
```

```
a.mean()
```

```
11.0
```

```
np.floor(a/10)
```

```
array([0., 0., 0., 0., 1., 1., 1., 1., 1., 2.])
```

```
c = np.concatenate((a, b))
```

```
c
```

```
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18, 20,  2,  4,  6,  8, 10, 12, 14,
       16, 18, 20])
```

```
a.sum()
```

```
110
```

```
np.std(a)
```

```
5.744562646538029
```

```
# Matrices 2x2
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])

# Multiplication de matrices (produit matriciel)
result = np.dot(matrix1, matrix2)
print(result)  # [[19, 22], [43, 50]]

# Transposition d'une matrice
transpose = np.transpose(matrix1)
print(transpose)  # [[1, 3], [2, 4]]
```

```
[[19 22]
 [43 50]]
[[1 3]
 [2 4]]
```

```
np.unique(c)
```

```
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18, 20])
```

```
matrix1
```

```
array([[1, 2],
       [3, 4]])
```

```
reversed_arr_columns = np.flip(matrix1, axis=1)
```

```
reversed_arr_columns
```

```
array([[2, 1],
       [4, 3]])
```

```
reversed_arr_lines = np.flip(matrix1, axis=0)
```

```
reversed_arr_lines
```

```
array([[3, 4],
       [1, 2]])
```

```
matrix1.flatten()
```

```
array([1, 2, 3, 4])
```

```
np.save('file1', a)
```

```
loaded_array = np.load('file1.npy')
```

```
loaded_array
```

```
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18, 20])
```

Double-cliquez (ou appuyez sur Entrée) pour modifier

```
from google.colab import drive
drive.mount('/content/drive')
```